# SJSU CMPE 172 TERM PROJECT SPRING 2026

In this term project, each student will design and implement a small enterprise-style Java / Spring Boot system for an Online Appointment Scheduling System.

The system represents a realistic enterprise backend with clear architectural layering, database persistence, concurrency handling, and integration boundaries.

Your system must support:

- Browsing available time slots
- Booking, canceling, and viewing appointments
- Provider-side management of availability
- Prevention of double-booking through concurrency control
- Integration with a mock external service (e.g., notification or calendar)

Spring Boot is used as the execution environment and integration platform. The primary grading emphasis is on system architecture, database design, transaction correctness, and enterprise-style behavior.

**Required Architectural Characteristics**

Your implementation must demonstrate:

- Spring Boot web controllers and HTTP endpoints
- Layered enterprise architecture
  Controller → Service → Repository → Database
- SQL schema design and JDBC-based persistence
- Explicit transaction management and ACID reasoning
- Concurrency control for booking conflicts
- A clear distribution boundary (mock remote service)
- Logging and basic system observability

**Technology Constraints**

- Required
  - Java + Spring Boot
  - SQL database accessed via JDBC (no ORM)
- Database (choose one)
  - MySQL
  - PostgreSQL
  - SQLite
- Not Allowed
  - ORM frameworks (Hibernate / JPA)
  - Auto-generated SQL
  - Non-Java backends

**Project Milestones**

- M1 - Project Proposal:
  - In this milestone, you will define the scope of your Online Appointment Scheduling System implemented in Java + Spring Boot.

SJSU CMPE 172 Spring 2026

- Requirements: 1 - 2 page PDF document including:
    - Your chosen scenario (e.g., clinic, medical office, salon, tutoring service, academic advising, fitness training, equipment rental, or similar appointment-based services)
    - User roles (e.g., customer, provider/admin)
    - At least 5 core features (e.g., browse available time slots, book appointments, cancel or reschedule appointments, view appointment history, provider availability management)
    - Expected main entities (users, providers, services, availability slots, appointments)
    - A simple block diagram of the system (client → Spring Boot app → DB)
    - A statement confirming you will use Java + Spring Boot + SQL DB
- Submission: 11:59pm on 2/4
    - Upload a PDF CMPE172_M1_Proposal_FirstName_LastName.pdf

- M2 – Database Design (ER + Relational Schema):
    - Design the **logical and relational structure** of your appointment system database using concepts from **ER/EER modeling**
    - Requirements:
        - ER or EER diagram (PDF or image)
        - Relational schema derived from your ER model, including:
            - users, providers, services, availability_slots, appointments
        - Clear primary keys, foreign keys, and relationship explanations
        - Brief justification (½ page) of key design decisions (cardinality, weak entities, constraints)
    - Submission: 11:59pm on 2/25
        - Upload Zip CMPE172_M2_Design_FirstName_LastName.zip containing:
            - ER diagram (.pdf)
            - Relational schema description(.pdf)
- M3 – Web Interface + Application Skeleton:
    - Implement a basic web interface for your system using Java + Spring Boot to demonstrate correct layered architecture and request flow. The interface may be server-rendered (templates) or return basic HTTP responses. Focus on **clear controller endpoints**, **request → service → repository flow**, and **correct interaction with the data layer**.
    - Requirements:
        - Wireframes or mockups for:
            - Home page
            - View available slots
            - Book appointment (form)
            - Confirmation page
        - Implement at least 2 working endpoints/pages in Spring Boot:
            - Example: GET / shows basic homepage
            - Example: GET /appointments lists appointments (mock or from DB)
        - Brief explanation (half-page) of your chosen pattern:
            - Page Controller vs Front Controller in your design
    - Submission: 11:59pm on 3/11
        - Upload a Zip CMPE172_M3_Web_FirstName_LastName.zip containing:
            - Source code for controllers/views
            - Wireframes (PDF or images)

SJSU CMPE 172 Spring 2026

        - A brief explanation (approximately half a page) describing how an incoming request flows through your application layers (controller → service → repository → database) (M3_DesignNotes.pdf).
- <u>M4 - Concurrency Scenario + Transaction Design:</u>
    - Design how your system prevents double booking of appointments using transactions and concurrency control
    - Requirements:
        - Identify a concrete race-condition scenario involving two users booking the same slot
        - Explain how ACID applies to this use case
        - Choose appropriate isolation levels and explain why
        - Provide a brief design explanation (1 page):
            - How booking/versioning/locking works
            - Use of @Transactional and version check / retry logic
    - Submission: 11:59pm on 3/27
        - PDF document: CMPE172_M4_Concurrency_FirstName_LastName.pdf
        - Include supporting Java and/or SQL snippets in an appendix that demonstrate your proposed concurrency control design
- <u>M5 - Distribution Boundary Design:</u>
    - Design a **distributed boundary** for your system by introducing a mock external service (e.g., notification or calendar service). Focus on **coarse-grained service interfaces**, clear responsibility boundaries, and interaction between components.
    - Requirements:
        - Choose one external service:
            - Notification service (send confirmation)
            - Provider calendar integration (mock)
        - Draw a distribution diagram (PDF or image):
            - Client → Appointment Scheduler → External Service
        - Implement:
            - One REST endpoint in your main app that calls the mock external service (can be simulated or stubbed)
        - Another Spring controller "pretending" to be the external service
        - Describe how API is coarse-grained and justify your design
    - Submission: 11:59pm on 4/22
        - Upload CMPE172_M5_Dist_FirstName_LastName.zip with:
            - Distribution diagram (.pdf or .png)
            - Short explanation PDF (M5_DesignNotes.pdf)
- <u>M6 - System Management + Logging Plan:</u>
    - Design how your system will be monitored and managed in production-like environments.
    - Requirements:
        - Logging plan:
            - What to log (key events/errors)?
            - Example log messages (info/warn/error)
        - At least one monitoring metric, e.g.:
            - Number of bookings per hour

SJSU CMPE 172 Spring 2026

        - Failed bookings
        - Average latency of booking operation
    - One failure scenario + recovery strategy
    - Design a health check endpoint (e.g., GET /health) and state what it returns
  - Submission: 11:59pm on 4/29
    - Upload CMPE172_M6_SysMGMT_FirstName_LastName.pdf
    - Include small illustrative code snippets in an appendix (e.g., logging statements, metric collection, or health check endpoint)
- Final Project – Online Appointment Scheduling System:
  - Submit your completed individual project
  - Requirements:
    - Working Application (Code)
      - Java + Spring Boot project
      - Uses SQL DB via JDBC
      - Includes layered architecture (Controller, Service, Repository)
    - Implements:
      - Booking flow
      - Double-booking prevention
      - Mock remote service call
      - Logging + health endpoint
    - Final Report (5 – 10 pages, PDF)
      - System overview
      - Architecture diagram
      - DB schema and rationale
      - Concurrency & transaction design
      - Distribution boundary design
      - System management & logging
      - Limitations & future improvements
    - Demo Video (5 - 10 minutes)
      - Show the app running locally
      - Demonstrate booking, concurrency handling, and remote boundary behavior
  - Submission: 11:59pm on 5/8
    - Canvas submission with:
      - GitHub repo link
      - Final PDF report
      - Demo video link (YouTube unlisted or similar)

SJSU CMPE 172 Spring 2026